



# SPARK EXECUTION FLOW

What REALLY Happens After `df.show()`

SparkSession

DAG

Catalyst

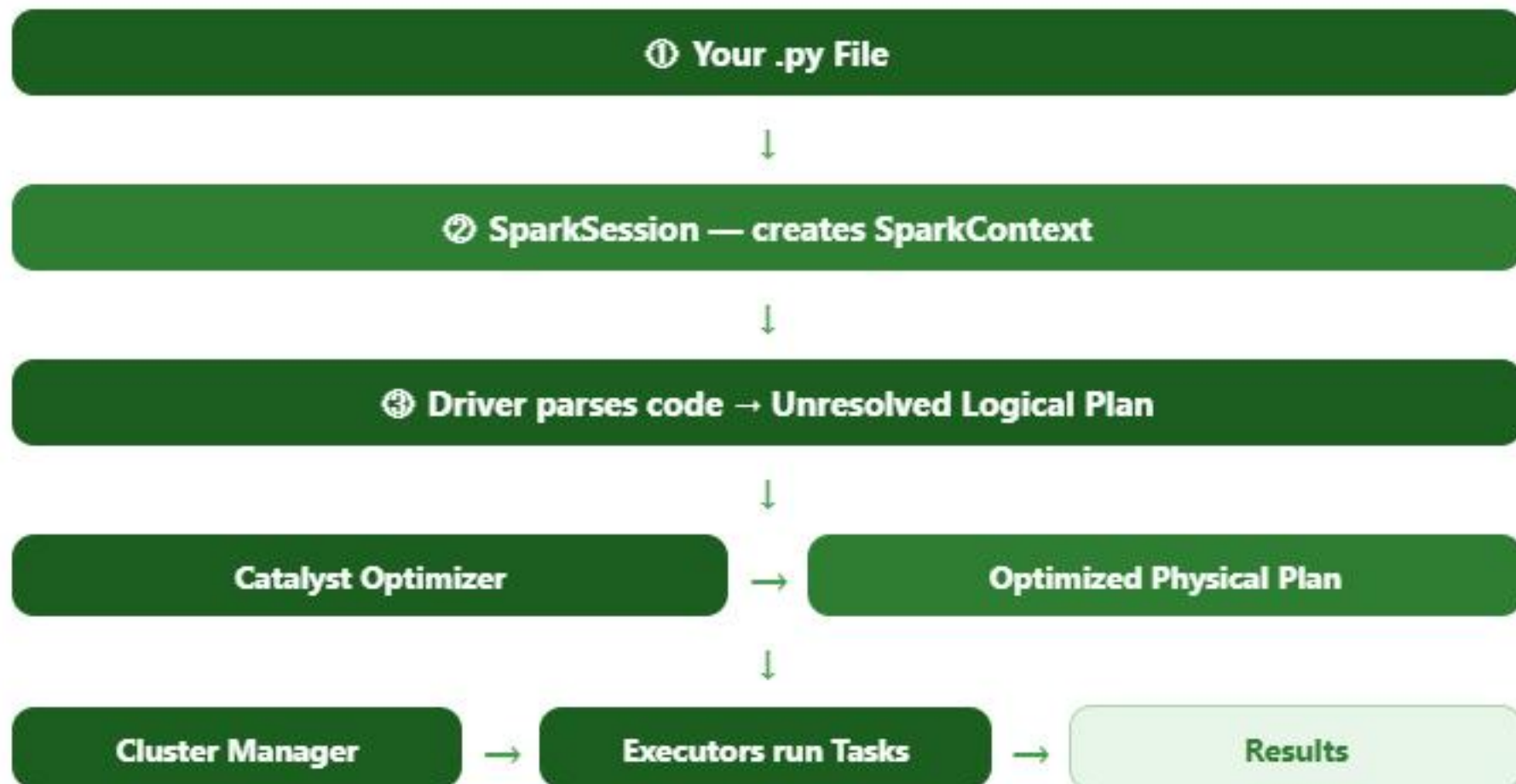
Executors

Stages & Tasks

**Interview Ready in 7 Slides**

Swipe to start learning →

*One `df.show()` triggers this entire chain*



**Key Interview Point:** The Driver plans everything. Executors process all data. The Driver never touches a single row of your dataset.

**S SPARKSESSION — ENTRY POINT (SPARK 2.0+)**

```
spark = (SparkSession .builder .appName("MyApp")  
        .config("spark.sql.shuffle.partitions","200") .getOrCreate())
```

One SparkSession per application. Unified replacement for SparkContext, SQLContext, HiveContext.

**Driver Responsibilities — 3 Core Jobs****① SPARKCONTEXT**

Connects to cluster.  
Negotiates resources from  
Cluster Manager.

**② DAG SCHEDULER**

Converts code into Jobs,  
Stages, and Tasks based on  
transformations.

**③ TASK SCHEDULER**

Sends each Task to an  
Executor on a Worker Node.

**Interview Tip:** "The Driver JVM runs your .py file. But it only *plans* — it never reads or shuffles data. All heavy lifting is in the Executors."

**Remember:** `spark.sparkContext` gives you the underlying SparkContext from a SparkSession.

## L LAZY EVALUATION — BUILD THE PLAN. RUN NOTHING.

### TRANSFORMATIONS (LAZY — NO DATA MOVES)

`.filter()`

`.select()`

`.groupBy()`

`.withColumn()`

`.join()`

### ACTIONS (TRIGGER FULL EXECUTION)

`.show()`

`.count()`

`.collect()`

`.write()`

## Catalyst Optimizer — 4 Phases

1 **Unresolved Logical Plan** — raw parse of your code



2 **Analyzed Logical Plan** — validates columns/types against catalog



3 **Optimized Logical Plan** — pushes filters down, prunes columns



4 **Physical Plan** — cost model selects best execution strategy

**Interview Tip:** "Catalyst is why DataFrames often outperform hand-written RDD code — it optimizes your logic *before* any data moves."



**C CLUSTER MANAGER — RESOURCE ALLOCATOR**

Decides how many Executors to start and how much CPU/memory each gets. The Driver asks; the Manager delivers.

- YARN
- Kubernetes
- Standalone
- Mesos

**Worker Nodes + Executors + Tasks**



**Interview Tip:** "Each Executor is a JVM process. Each Task inside it processes exactly **one partition**. More partitions = more parallelism."

Use `.cache()` to store a DataFrame in Executor memory — avoids recomputing the same data across multiple Actions.

How Spark decomposes your code into parallel units

**D** DAG — DIRECTED ACYCLIC GRAPH

A complete map of all your transformations. Built before any data moves. No cycles — always forward. The DAG Scheduler uses this to generate Stages.

**JOB**

Created by 1 Action (e.g. df.show())

**Stage 1**

No shuffle yet

**Task 1**  
Partition 1

**Task 2**  
Partition 2

**Task 3**  
Partition 3

↑ SHUFFLE — data redistributed across the network (expensive)

**Stage 2**

After shuffle

**Task 4**  
Partition 1

**Task 5**  
Partition 2

**Task 6**  
Partition 3

**Interview Tip:** "Every `groupBy()`, `join()` on unpartitioned data, and `distinct()` causes a shuffle. Minimize them — shuffles are the #1 Spark performance killer."

Save this slide before your next Data Engineering interview

| COMPONENT    | ROLE                | KEY INTERVIEW FACT                              |
|--------------|---------------------|-------------------------------------------------|
| SparkSession | Entry point         | One per app                                     |
| Driver       | Plans + coordinates | Never processes data                            |
| Catalyst     | Optimizes plans     | 4 phases: parse → analyze → optimize → physical |
| Job          | 1 per Action        | df.show() = 1 Job                               |
| Stage        | Shuffle boundary    | Minimize shuffles                               |
| Task         | 1 per Partition     | = unit of parallelism                           |
| Executor     | Runs Tasks          | JVM on Worker Node                              |

Follow for daily Data Engineering content

@chintapradeep

Spark · Python · SQL · Interview Prep